POST-MORTEM TÉCNICO

Despliegue de Aplicación Serverless en AWS

De cero a producción — Lecciones reales de troubleshooting

Abril 2026 | AWS Amplify · API Gateway · Lambda · DynamoDB · Cognito · S3

1. Contexto del Proyecto

Se desplegó una aplicación web fullstack llamada Invoice Digitizer — un digitalizador de recibos con React en el frontend y arquitectura serverless en el backend usando los siguientes servicios de AWS:

Servicio	Rol en la arquitectura
AWS Amplify	Hosting del frontend React
API Gateway	Capa de entrada HTTP (REST)
AWS Lambda	Lógica de negocio (3 funciones Python)
DynamoDB	Base de datos de recibos procesados
Amazon S3	Almacenamiento de imágenes originales y resultados
Amazon Textract	OCR para extracción de datos del recibo
Amazon Cognito	Autenticación de usuarios
Amazon SNS	Notificaciones por email al procesar

2. Línea de Tiempo de Errores

El proceso de troubleshooting atravesó 6 fases distintas, cada una con su propio error raíz:

Fase 1 — Error de Build en AWS Amplify

```
npm error: lock file's typescript@6.0.2 does not satisfy typescript@4.9.5
```

¿Qué pasó?

El archivo package-lock.json generado en Codespaces (Node v24) era incompatible con el entorno de build de Amplify (Node v18/20). npm ci es estricto: exige sincronización exacta entre package.json y package-lock.json.

Error adicional descubierto

Amplify tenía una configuración de build guardada en la consola que sobreescribía el amplify.yml del repositorio. Todos los cambios al archivo eran ignorados silenciosamente.

Solución

- Cambiar npm ci por npm install en amplify.yml
- Forzar Node 20 con nvm use 20 en la fase preBuild
- Usar --prefix en lugar de cd para evitar problemas de directorio entre fases
- Limpiar la configuración guardada en la consola de Amplify

Fase 2 — Error CORS en API Gateway

```
Access to fetch blocked by CORS policy: No 'Access-Control-Allow-Origin'
header
```

¿Qué pasó?

La variable de entorno FRONTEND_URL no estaba configurada en las funciones Lambda. Python devolvía None como valor del header, que el browser rechazaba literalmente.

Solución

- Agregar variable de entorno FRONTEND_URL en cada Lambda
- Agregar fallback en el código: os.environ.get('FRONTEND_URL', 'https://...')
- Habilitar CORS en API Gateway para todos los recursos
- Redesplegar la API en etapa prod (paso crítico frecuentemente olvidado)

Fase 3 — DurableConfig: el error más difícil

```
You cannot invoke a durable function using an unqualified ARN.
```

¿Qué pasó?

Las funciones Lambda fueron creadas con DurableConfig activo (RetentionPeriodInDays: 14, ExecutionTimeout: 900). Esta configuración exige que toda invocación use un ARN completamente calificado con versión numérica — no $LATEST ni el nombre simple.

Por qué fue difícil

El error aparecía en tres capas diferentes (CLI, API Gateway, browser) con mensajes ligeramente distintos, lo que dificultaba identificar la causa raíz. Además, $LATEST se interpretaba como variable de shell vacía al ejecutar comandos.

Solución

- Recrear las Lambdas desde cero usando Python 3.12 en lugar de 3.14
- Descargar el código con aws lambda get-function y crear nueva función con create-function
- Actualizar las integraciones de API Gateway para apuntar a las nuevas funciones _v2

Fase 4 — Permisos de IAM insuficientes

```
AccessDeniedException: not authorized to perform: dynamodb:Query
```

¿Qué pasó?

El rol LambdaVoice solo tenía AWSLambdaBasicExecutionRole. Las nuevas Lambdas no podían acceder a DynamoDB ni a S3.

Solución

- Adjuntar AmazonDynamoDBFullAccess al rol LambdaVoice
- Adjuntar AmazonS3FullAccess al rol LambdaVoice

Fase 5 — Permisos de invocación de Lambda

```
Execution failed due to configuration error: Invalid permissions on Lambda
function
```

¿Qué pasó?

Al recrear las Lambdas con nombres _v2, los permisos de resource-based policy que permiten a API Gateway invocarlas no se transfirieron automáticamente. La integración apuntaba a la nueva función pero sin permiso de invocación.

Solución

- Ejecutar aws lambda add-permission para cada función _v2
- Especificar el source-arn correcto con el método HTTP y recurso exacto
- Redesplegar la API después de cada cambio de permisos

Fase 6 — Cognito USER_PASSWORD_AUTH deshabilitado

```
InvalidParameterException: USER_PASSWORD_AUTH flow not enabled for this
client
```

¿Qué pasó?

Para probar el endpoint con un token real desde CloudShell, se necesitaba autenticar directamente con Cognito. El cliente de Cognito no tenía habilitado el flujo de autenticación por usuario y contraseña.

Solución

- Ejecutar aws cognito-idp update-user-pool-client con los explicit-auth-flows correctos
- Solo necesario para pruebas desde CLI — el frontend usa SRP que sí estaba habilitado

---

3. Métodos de Diagnóstico Utilizados

3.1 Logs de Build de Amplify

El panel de Implementaciones de Amplify muestra logs línea por línea del proceso de build. Método clave: hacer clic en la fase que falla (Compilar o Implementar) para expandir los logs negros. Las últimas 10 líneas siempre contienen el error real.

3.2 Invocación directa de Lambda desde CLI

```
aws lambda invoke --function-name NOMBRE --payload '{...}' response.json -
-log-type Tail
```

Permite aislar si el error está en Lambda o en la capa de API Gateway. Si la invocación directa funciona pero API Gateway falla, el problema está en la configuración de la integración o permisos.

3.3 CloudWatch Logs — API Gateway Execution Logs

Habilitado con update-stage usando loglevel=INFO. Registra cada paso del ciclo de vida de un request: autorización, integración, respuesta. El mensaje clave que resolvió la Fase 5:

```
Execution failed due to configuration error: Invalid permissions on Lambda
function
```

3.4 CloudWatch Logs Insights

Herramienta de consulta SQL-like sobre logs. Útil para filtrar por error, request ID o rango de tiempo. Query de ejemplo:

```
fields @timestamp, @message | filter @message like /upload-url/ | sort
@timestamp desc | limit 20
```

3.5 aws apigateway get-integration

Verifica a qué Lambda apunta exactamente cada método de API Gateway. En varias ocasiones la integración apuntaba a la función antigua sin DurableConfig fix. Este comando fue decisivo:

```
aws apigateway get-integration --rest-api-id sl2rrcev1b --resource-id ID -
-http-method POST --query 'uri'
```

3.6 aws lambda list-versions-by-function

Reveló la existencia de DurableConfig en las funciones, que era la causa raíz del error de ARN no calificado. Sin este comando, el error habría sido imposible de diagnosticar.

---

4. Lecciones Aprendidas

4.1 Sobre AWS Amplify

- ⚠ La consola de Amplify tiene prioridad sobre el amplify.yml del repo. Si ambos existen, la consola gana siempre.
- ⚠ Cada fase de build (preBuild, build) empieza desde la raíz del repo. El cd de preBuild no persiste.
- ✅ Usar --prefix en lugar de cd es más robusto para proyectos en subdirectorios.

- ✅ npm ci es estricto por diseño. En CI/CD con múltiples entornos de Node, npm install es más seguro.

4.2 Sobre API Gateway y Lambda

- ⚠️ Siempre hacer Implementar API después de cualquier cambio. Sin este paso nada aplica.
- ⚠️ Los permisos de resource-based policy de Lambda no se heredan al recrear funciones con nombres distintos.
- ⚠️ La versión de Python importa: Python 3.14 activó DurableConfig automáticamente. Python 3.12 no.
- ✅ Separar el diagnóstico en capas: ¿el error está en Lambda, en API Gateway, o en el Authorizer?

4.3 Sobre IAM y Permisos

- ⚠️ Los roles de Lambda necesitan permisos explícitos para cada servicio: DynamoDB, S3, SNS, Textract.
- ⚠️ AWSLambdaBasicExecutionRole solo da permisos de logging. No da acceso a ningún otro servicio.
- ✅ Los cambios de IAM tardan segundos en propagarse. Siempre esperar antes de volver a probar.

4.4 Sobre CORS

- ⚠️ CORS tiene dos capas: API Gateway (OPTIONS/headers) y Lambda (headers en el return).
- ⚠️ Si la Lambda devuelve None en Access-Control-Allow-Origin, el browser lo rechaza igual que si no existiera.
- ✅ Siempre usar fallback: os.environ.get('FRONTEND_URL', 'https://tu-app.amplifyapp.com')

4.5 Sobre Metodología de Troubleshooting

- ✅ Aislar la capa del error primero: browser → API Gateway → Lambda → servicio AWS.
- ✅ Invocar Lambda directamente antes de probar via API Gateway para aislar responsabilidades.
- ✅ Los logs de CloudWatch son la fuente de verdad. Todo lo demás es suposición.

- ✅ Cuando un fix no funciona, verificar que el cambio realmente se aplicó antes de buscar otra causa.

---

5. Comandos Clave del Proceso

Propósito	Comando
Ver permisos de Lambda	aws lambda get-policy --function-name NOMBRE
Verificar integración API Gateway	aws apigateway get-integration --rest-api-id ID --resource-id RID --http-method GET
Agregar permiso a Lambda	aws lambda add-permission --function-name NOMBRE --principal apigateway.amazonaws.com ...
Redesplegar API	aws apigateway create-deployment --rest-api-id ID --stage-name prod
Ver versiones y DurableConfig	aws lambda list-versions-by-function --function-name NOMBRE
Log de invocación	aws lambda invoke ... --log-type Tail --query 'LogResult' | base64 -d
Permisos del rol IAM	aws iam list-attached-role-policies --role-name ROL
Agregar política IAM	aws iam attach-role-policy --role-name ROL --policy-arn arn:aws:iam::aws:policy/...
Obtener token Cognito	aws cognito-idp initiate-auth --auth-flow USER_PASSWORD_AUTH ...
Probar endpoint	curl -X POST URL -H 'Authorization: TOKEN' -d '{"content_type":"image/jpeg"}'

---

6. Estado Final del Sistema

Después de resolver todos los errores, el flujo completo funciona así:

1. Usuario sube imagen desde el frontend en Amplify
2. Cognito autentica al usuario y emite un JWT
3. API Gateway valida el JWT con el Authorizer de Cognito
4. get_upload_url_v2 genera una URL presignada de S3
5. El frontend sube la imagen directamente a S3 con la URL presignada

6. S3 dispara la Lambda process_invoice via trigger de eventos
7. Textract procesa la imagen y extrae datos del recibo
8. Los datos se guardan en DynamoDB y SNS envía notificación por email
9. get_invoices_v2 retorna el historial al dashboard del usuario

---

"El debugging no es un fracaso — es el proceso de aprender dónde estaban tus suposiciones incorrectas."

Documento generado: Abril 2026